Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

# Лабораторная работа №2 по курсу

# «Операционные системы»

Группа: М80-211БВ-25

Студент: Князев Александр Владимирович

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 29.10.26

Москва, 2026

# Постановка задачи

**Вариант 6**

Разработать программу на языке Си для перемножения двух матриц, содержащих комплексные числа, с использованием потоков операционной системы. Цель работы - получить практические навыки управления потоками и согласования их работы с общей памятью.

Максимальное количество одновременно существующих потоков задаётся аргументом запуска. Требуется показать число потоков стандартными средствами ОС и исследовать зависимость ускорения и эффективности от размеров входных матриц и числа потоков.

Номер варианта определён по номеру 6 в списке группы: ((6 - 1) % 20) + 1 = 6.

# Общий метод и алгоритм решения

**Представление данных**

Комплексное число хранится в структуре Complex с двумя полями double: real и imag. Структура Matrix содержит число строк rows, число столбцов cols и указатель data на массив указателей на строки. Для матрицы из m строк выделяется m + 1 блок памяти: один массив указателей и m массивов элементов.

Для A размера m×n и B размера n×k получается C размера m×k. Ввод выполняется в порядке: размеры A, элементы A, размеры B, элементы B. Каждый элемент задаётся двумя конечными числами: действительной и мнимой частями.

**Формулы вычисления**

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i$$

$$C_{i,j} = \sum_{t=0}^{n-1} A_{i,t} \cdot B_{t,j}$$

Функция multiply_rows() вычисляет строки результата в диапазоне от start_row включительно до end_row исключительно. Для каждого элемента результата выполняется последовательное суммирование произведений по общей размерности n. Сложность алгоритма составляет O(mnk), дополнительная память для матриц - O(mn + nk + mk).

**Распределение строк между потоками**

При запуске ./main N число вычисляющих потоков p равно min(N, m), включая основной поток. Создаётся p - 1 дополнительных потоков, а основной вычисляет последний диапазон строк. При p = 1 дополнительные потоки не создаются.

Каждому потоку назначается m / p строк; первые m % p потоков получают по одной дополнительной строке. Например, для восьми строк и трёх потоков диапазоны равны [0, 3), [3, 6), [6, 8). Структура ThreadData содержит адреса A, B и C и границы назначенного диапазона.

A и B во время вычислений только читаются. Каждый элемент C записывает ровно один поток, поэтому мьютекс для записи элементов не требуется. Основной поток ожидает завершения дополнительных потоков, затем выводит результат и освобождает память. Это ожидание обеспечивает согласование завершения вычислений с последующим использованием матрицы.

**Использованные средства ОС и библиотеки**

pthread_create() создаёт дополнительный поток и запускает worker() с адресом его ThreadData. pthread_join() ожидает завершения указанного потока. Эти функции принадлежат библиотечному интерфейсу POSIX Threads; их не следует смешивать с непосредственными системными вызовами ядра.

clock_gettime(CLOCK_MONOTONIC, ...) измеряет прошедшее время монотонными часами. calloc(), malloc() и free() используются для динамической памяти. При ошибке создания потока программа ожидает уже созданные потоки и завершает работу без вывода частичного результата.

# Код программы

**main.c**

Полный исходный код программы приведён в одном файле main.c.

```
#define _POSIX_C_SOURCE 200809L

#include <errno.h>
#include <limits.h>
#include <math.h>
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

struct Complex {
    double real;
    double imag;
};

struct Matrix {
    int rows;
    int cols;
    struct Complex **data;
};

struct ThreadData {
    const struct Matrix *a;
    const struct Matrix *b;
    struct Matrix *c;
    int start_row;
    int end_row;
};

void free_matrix(struct Matrix *matrix)
{
    if (matrix->data == NULL) {
        return;
    }

    for (int row = 0; row < matrix->rows; row++) {
        free(matrix->data[row]);
    }

    free(matrix->data);
    matrix->data = NULL;
}

int create_matrix(struct Matrix *matrix, int rows, int cols)
{
    matrix->rows = rows;
```

```
    matrix->cols = cols;

    if ((size_t)rows > SIZE_MAX / sizeof(struct Complex *) ||
        (size_t)cols > SIZE_MAX / sizeof(struct Complex)) {
        return 0;
    }

    matrix->data = calloc((size_t)rows, sizeof(struct Complex *));
    if (matrix->data == NULL) {
        return 0;
    }

    for (int row = 0; row < rows; row++) {
        matrix->data[row] = malloc((size_t)cols * sizeof(struct Complex));
        if (matrix->data[row] == NULL) {
            return 0;
        }
    }

    return 1;
}

int read_matrix(struct Matrix *matrix)
{
    int rows;
    int cols;

    if (scanf("%d %d", &rows, &cols) != 2 || rows < 1 || cols < 1) {
        return 0;
    }

    if (!create_matrix(matrix, rows, cols)) {
        return 0;
    }

    for (int row = 0; row < matrix->rows; row++) {
        for (int col = 0; col < matrix->cols; col++) {
            if (scanf("%lf %lf", &matrix->data[row][col].real,
                      &matrix->data[row][col].imag) != 2) {
                return 0;
            }

            if (!isfinite(matrix->data[row][col].real) ||
                !isfinite(matrix->data[row][col].imag)) {
                return 0;
            }
        }
    }

    return 1;
}

void print_matrix(const struct Matrix *matrix)
{
    for (int row = 0; row < matrix->rows; row++) {
        for (int col = 0; col < matrix->cols; col++) {
            printf("(%.12g%+.12gi) ", matrix->data[row][col].real,
                   matrix->data[row][col].imag);
        }
        printf("\n");
    }
}
```

```
void multiply_rows(const struct Matrix *a, const struct Matrix *b,
                   struct Matrix *c, int start_row, int end_row)
{
    for (int row = start_row; row < end_row; row++) {
        for (int col = 0; col < b->cols; col++) {
            c->data[row][col].real = 0;
            c->data[row][col].imag = 0;

            for (int k = 0; k < a->cols; k++) {
                struct Complex product;

                product.real = a->data[row][k].real * b->data[k][col].real
                             - a->data[row][k].imag * b->data[k][col].imag;

                product.imag = a->data[row][k].real * b->data[k][col].imag
                             + a->data[row][k].imag * b->data[k][col].real;

                c->data[row][col].real += product.real;
                c->data[row][col].imag += product.imag;
            }
        }
    }
}

void *worker(void *argument)
{
    struct ThreadData *data = argument;

    multiply_rows(data->a, data->b, data->c, data->start_row, data->end_row);

    return NULL;
}

int multiply_parallel(const struct Matrix *a, const struct Matrix *b,
                      struct Matrix *c, int thread_count)
{
    if (thread_count == 1) {
        multiply_rows(a, b, c, 0, a->rows);
        return 1;
    }

    if ((size_t)thread_count > SIZE_MAX / sizeof(struct ThreadData) ||
        (size_t)(thread_count - 1) > SIZE_MAX / sizeof(pthread_t)) {
        return 0;
    }

    pthread_t *threads = malloc((size_t)(thread_count - 1) *
sizeof(pthread_t));
    struct ThreadData *tasks = malloc((size_t)thread_count * sizeof(struct
ThreadData));

    if (threads == NULL || tasks == NULL) {
        free(threads);
        free(tasks);
        return 0;
    }

    int rows_per_thread = a->rows / thread_count;
    int extra_rows = a->rows % thread_count;
    int next_row = 0;

    for (int index = 0; index < thread_count; index++) {
        tasks[index].a = a;
        tasks[index].b = b;
        tasks[index].c = c;
        tasks[index].start_row = next_row;
```

```
        next_row += rows_per_thread;
        if (index < extra_rows) {
            next_row++;
        }

        tasks[index].end_row = next_row;
    }

    int created = 0;
    int create_error = 0;

    for (int index = 0; index < thread_count - 1; index++) {
        create_error = pthread_create(&threads[index], NULL, worker,
&tasks[index]);
        if (create_error != 0) {
            break;
        }
        created++;
    }

    if (create_error == 0) {
        struct ThreadData *main_task = &tasks[thread_count - 1];
        multiply_rows(a, b, c, main_task->start_row, main_task->end_row);
    }

    for (int index = 0; index < created; index++) {
        int join_error = pthread_join(threads[index], NULL);
        if (join_error != 0) {
            fprintf(stderr, "Ошибка ожидания потока: %s\n",
strerror(join_error));
            exit(EXIT_FAILURE);
        }
    }

    free(threads);
    free(tasks);

    if (create_error != 0) {
        fprintf(stderr, "Ошибка создания потока: %s\n",
strerror(create_error));
        return 0;
    }

    return 1;
}

int read_thread_limit(const char *text)
{
    char *end;
    errno = 0;
    long value = strtol(text, &end, 10);

    if (errno != 0 || end == text || *end != '\0' || value < 1 || value >
INT_MAX) {
        return 0;
    }

    return (int)value;
}

double get_time(void)
{
    struct timespec time;

    if (clock_gettime(CLOCK_MONOTONIC, &time) != 0) {
        perror("Не удалось измерить время");
        exit(EXIT_FAILURE);
```

```
    }

    return (double)time.tv_sec + (double)time.tv_nsec / 1000000000.0;
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "Запуск: %s число_потоков\n", argv[0]);
        return EXIT_FAILURE;
    }

    int thread_limit = read_thread_limit(argv[1]);
    if (thread_limit == 0) {
        fprintf(stderr, "Количество потоков должно быть целым числом от 1 до
%d.\n", INT_MAX);
        return EXIT_FAILURE;
    }

    struct Matrix a = {0};
    struct Matrix b = {0};
    struct Matrix c = {0};

    printf("Введите размеры первой матрицы (строки столбцы), затем её элементы
по строкам.\n"
           "Элемент: действительная и мнимая части. Например, 3 -2 означает 3-
2i.\n");
    fflush(stdout);

    if (!read_matrix(&a)) {
        fprintf(stderr, "Не удалось прочитать первую матрицу или выделить
память.\n");
        free_matrix(&a);
        return EXIT_FAILURE;
    }

    printf("Введите размеры второй матрицы (строки столбцы), затем её элементы
по строкам:\n");
    fflush(stdout);

    if (!read_matrix(&b)) {
        fprintf(stderr, "Не удалось прочитать вторую матрицу или выделить
память.\n");
        free_matrix(&a);
        free_matrix(&b);
        return EXIT_FAILURE;
    }

    if (a.cols != b.rows) {
        fprintf(stderr, "Число столбцов первой матрицы должно равняться числу
строк второй.\n");
        free_matrix(&a);
        free_matrix(&b);
        return EXIT_FAILURE;
    }

    if (!create_matrix(&c, a.rows, b.cols)) {
        fprintf(stderr, "Не удалось выделить память для результата.\n");
        free_matrix(&a);
        free_matrix(&b);
        free_matrix(&c);
        return EXIT_FAILURE;
    }

    int thread_count = thread_limit;
    if (thread_count > a.rows) {
        thread_count = a.rows;
```

```
    }

    double start = get_time();
    int success = multiply_parallel(&a, &b, &c, thread_count);
    double elapsed = get_time() - start;

    if (success) {
        printf("Время умножения: %.9f c\n", elapsed);
        printf("Результат:\n");
        print_matrix(&c);
    } else {
        fprintf(stderr, "Не удалось выполнить многопоточное умножение.\n");
    }

    free_matrix(&a);
    free_matrix(&b);
    free_matrix(&c);

    if (!success) {
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

# Протокол работы программы

**Условия и происхождение проверок**

Протокол получен автоматизированными запусками помощника при подготовке отчёта 29.10.2026. Исходные данные, результаты и сценарий повторного запуска сохранены вместе с отчётом в папке materials.

Окружение: Ubuntu 24.04.5 LTS, x86_64, glibc 2.39; процессор Intel Core i5-13420H; доступны 12 логических процессоров. Компилятор GCC 13.3.0. Приведённые времена относятся к этому окружению и не характеризуют автоматически другую систему.

Сборка выполнена следующей командой, без дополнительных флагов оптимизации:

```
gcc -std=c99 -Wall -Wextra -Wpedantic -Werror main.c -o main -lm
```

В этом окружении команда успешно собирает программу. Для других систем POSIX Threads может потребоваться флаг -pthread; изменение флагов сборки требует отдельной серии измерений.

**Пример с двумя матрицами второго порядка**

```
./main 1
```

Полный ввод:

```
2 2
1.0 1.0 2 0
3 0 0.0 1.0
2 2
1 0 0.0 1.0
2 0 1 0
```

Фрагмент фактического вывода после подсказок ввода:

```
Время умножения: 0.000002638 c
Результат:
(5+1i) (1+1i)
(3+2i) (0+4i)
```

Результат совпадает с ручным расчётом. Например, верхний правый элемент равен (1+i)i + 2 = 1+i.

**Пример с прямоугольной матрицей**

```
./main 2
```

Полный ввод для A размера 3×2 и B размера 2×2:

```
3 2
1.0 1.0 2 0
0.0 1.0 -1 0
3 0 2.0 -1.0
2 2
1 0 0.0 1.0
2.0 -1.0 1 0
```

Фрагмент фактического вывода:

```
Время умножения: 0.000307527 с
Результат:
(5-1i) (1+1i)
(-2+2i) (-2+0i)
(6-4i) (2+2i)
```

Дополнительный поток получает первые две строки, основной - третью. Результат совпадает с независимым расчётом через комплексные числа Python.

**Проверки корректности и ошибок**

Выполнена 31 проверка: 22 положительных запуска и 9 запусков с ошибочными параметрами или данными. В положительных тестах результат сравнивался с независимым вычислением средствами комплексной арифметики Python с допустимым абсолютным расхождением $10^{-9}$. Все проверки пройдены.

| Размер A | Размер B | Пределы потоков | Результат |
|---|---|---|---|
| 1×1 | 1×1 | 1, 2, 3, 8 | Совпадает |
| 1×3 | 3×4 | 1, 2, 3, 8 | Совпадает |
| 2×3 | 3×4 | 1, 2, 3, 8 | Совпадает |
| 7×3 | 3×5 | 1, 2, 3, 8 | Совпадает |
| 8×4 | 4×2 | 1, 2, 3, 8 | Совпадает |

Проверены отсутствие аргумента, нулевой и отрицательный предел потоков, текстовый аргумент, переполнение допустимого диапазона аргумента, нулевой размер, неполный ввод, NaN и несовместимые размеры. В каждом случае получен ненулевой код завершения; матрица частичного результата не выводилась.

**Трассировка потоков с помощью strace**

Для наблюдения за созданием потоков использована утилита strace 6.8. В коде дополнительные потоки создаются в multiply_parallel() вызовом pthread_create() внутри цикла, выполняющегося p - 1 раз. В наблюдавшемся запуске библиотека обратилась к ядру через системный вызов clone3().

Команда для повторения проверки из папки OC_Lab2:

```
strace -f -e trace=clone,clone3 -o trace.txt ./main 4
```

Ключ -f включает отслеживание создаваемых потоков, -e оставляет только системные вызовы clone и clone3, а -o сохраняет трассировку в файл. Ввод:

```
4 1
1 0
2 0
3 0
4 0
1 1
2 0
```

Умножается столбец из четырёх чисел на матрицу 1×1 со значением 2+0i. Результат содержит 2+0i, 4+0i, 6+0i и 8+0i. Четыре строки первой матрицы позволяют использовать указанный предел p = 4.

Основные строки фактической трассировки приведены ниже. Для читаемости в строках clone3 опущены адреса и часть флагов; идентификаторы и результаты вызовов сохранены. Полный вывод находится в materials/strace-threads.txt.

```
18166 clone3({flags=CLONE_VM|...|CLONE_THREAD|...}, 88) = 18167
18166 clone3({flags=CLONE_VM|...|CLONE_THREAD|...}, 88) = 18168
18167 +++ exited with 0 +++
18166 clone3({flags=CLONE_VM|...|CLONE_THREAD|...}, 88) = 18169
18168 +++ exited with 0 +++
18169 +++ exited with 0 +++
18166 +++ exited with 0 +++
```

Слева указан TID потока, выполнившего вызов. Основной поток 18166 создал потоки 18167, 18168 и 18169. Положительное значение после знака равенства является TID созданного потока и подтверждает успешное выполнение вызова.

Флаг CLONE_THREAD объединяет новый поток с вызывающим в одну группу потоков процесса. Флаг CLONE_VM означает совместное использование адресного пространства: потоки работают с общими матрицами A, B и C. Строки exited with 0 показывают нормальное завершение потоков.

Вычисления в этом примере короткие: поток 18167 завершился до создания потока 18169. Трассировка подтверждает создание трёх дополнительных потоков за время запуска. Одновременное существование четырёх потоков проверено отдельно на более крупных матрицах с помощью ps.

**Подтверждение числа потоков средствами Linux**

На матрицах 512×512 программа запускалась в фоне. Число потоков наблюдалось через /proc/PID/task, а ниже приведены реальные снимки ps. PID и TID относятся только к этим запускам.

```
./main 4 < input.txt > output.txt &
program_pid=$!
ps -L -p "$program_pid" -o pid,tid,stat,comm
wait "$program_pid"
```

Предел 1; максимальное наблюдавшееся количество потоков - 1.

```
PID     TID STAT COMMAND
    332     332 R    main
```

Предел 4; максимальное наблюдавшееся количество потоков - 4.

```
PID     TID STAT COMMAND
    334     334 Rl   main
    334     335 Rl   main
    334     336 Rl   main
    334     337 Rl   main
```

В запуске с пределом 4 наблюдались основной поток и три дополнительных. Потоки могут завершаться в разное время, поэтому к концу работы текущий показатель уменьшается. На маленьких матрицах наблюдение следует успеть выполнить до завершения программы.

**Исследование ускорения и эффективности**

Использовались квадратные матрицы порядка 16, 128 и 512. Все элементы обеих матриц равны 1+i; для каждого элемента результата ожидается 2ni. Правильность всей результирующей матрицы проверена после каждого измерительного запуска.

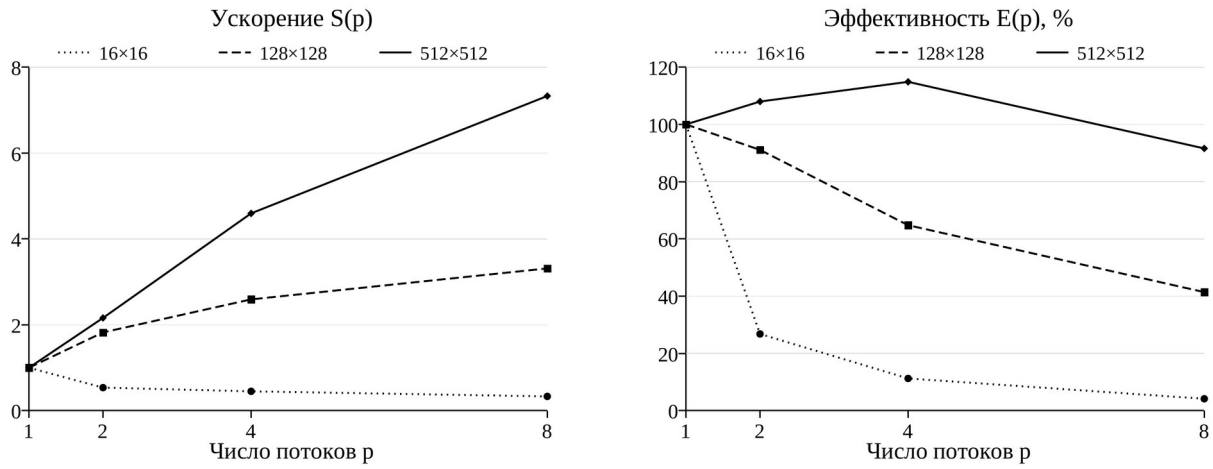
Для каждой пары размера и числа потоков выполнен один прогревочный запуск и пять измеряемых запусков. В каждом повторе порядок вариантов p = 1, 2, 4, 8 перемешивался с фиксированным начальным значением генератора. В таблице приведены медианы и диапазоны пяти значений. Всего выполнены 12 прогревочных и 60 измеряемых запусков.

T(p) измеряется внутри программы вокруг multiply_parallel(): в него входят подготовка заданий, создание потоков, вычисление и ожидание завершения. Ввод, выделение памяти матриц, вывод результата и освобождение матриц исключены. Потоки не закреплялись за ядрами; фоновая нагрузка и частоты процессора не фиксировались.

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}$$

Здесь p включает основной поток. Для процентов E(p) умножается на 100. Все размеры не меньше восьми, поэтому в этой серии фактическое количество вычисляющих потоков совпадает с указанным пределом.

| Размер | p | Медиана T, с | Мин - макс, с | S(p) | E(p), % |
|---|---|---|---|---|---|
| 16×16 | 1 | 0.000136384 | 0.000067 - 0.000149 | 1.000 | 100.0 |
| 16×16 | 2 | 0.000254574 | 0.000192 - 0.000414 | 0.536 | 26.8 |
| 16×16 | 4 | 0.000303217 | 0.000227 - 0.000560 | 0.450 | 11.2 |
| 16×16 | 8 | 0.000411434 | 0.000284 - 0.002171 | 0.331 | 4.1 |
| 128×128 | 1 | 0.041095008 | 0.033726 - 0.044631 | 1.000 | 100.0 |
| 128×128 | 2 | 0.022556163 | 0.017106 - 0.030554 | 1.822 | 91.1 |
| 128×128 | 4 | 0.015858289 | 0.009436 - 0.021007 | 2.591 | 64.8 |
| 128×128 | 8 | 0.012406507 | 0.009620 - 0.013658 | 3.312 | 41.4 |
| 512×512 | 1 | 3.600341781 | 3.420705 - 3.682567 | 1.000 | 100.0 |
| 512×512 | 2 | 1.667005257 | 1.580701 - 1.863126 | 2.160 | 108.0 |
| 512×512 | 4 | 0.783563249 | 0.740897 - 0.849334 | 4.595 | 114.9 |
| 512×512 | 8 | 0.491309703 | 0.488580 - 0.520272 | 7.328 | 91.6 |




Зависимость ускорения и эффективности от числа потоков

**Объяснение результатов**

Для матриц 16×16 переход от одного к восьми потокам увеличил медианное время с 0,000136384 до 0,000411434 с. Объём вычислений слишком мал, чтобы компенсировать подготовку потоков и ожидание их завершения. Малые времена также имеют заметный относительный разброс.

Для 128×128 восемь потоков дали ускорение 3,312 и эффективность 41,4%. Для 512×512 ускорение составило 7,328 при эффективности 91,6%. Рост числа арифметических операций увеличивает долю полезной работы относительно расходов на управление потоками.

Для 512×512 при двух и четырёх потоках расчётная эффективность составила 108,0% и 114,9%. Это сверхлинейные значения относительно измеренного однопоточного базового времени. Возможные причины - различия работы кэша, частот, назначения потоков на ядра и фоновой нагрузки. Без отдельного профилирования и фиксации условий нельзя установить вклад каждой причины или считать эти результаты гарантированной эффективностью алгоритма.

Следовательно, большее число потоков полезно прежде всего при достаточном объёме вычислений. Выбор лучшего p для конкретной системы требует измерений. Сравнивать следует одинаковые данные, одинаковую сборку и одинаковые границы измеряемого участка.

**Повторение экспериментов**

Исходные измерения, тесты и сведения об окружении сохранены в папке materials рядом с отчётом. Для повторения проверок и измерений из папки OC_Lab2 используется команда:

```
python3 report/materials/run_experiments.py
```

Полная трассировка сохранена в strace-threads.txt. После сборки main.c её можно повторить с сохранённым вводом strace-input.txt:

```
strace -f -e trace=clone,clone3 -o trace.txt ./main 4 <
report/materials/strace-input.txt
```

# Вывод

Разработана программа перемножения матриц комплексных чисел с ограничением общего числа потоков через аргумент запуска. Разделение результата по строкам позволяет вычислять независимые элементы без мьютекса, а pthread_join() согласует завершение потоков с выводом и освобождением памяти. Корректность результатов проверена тестами, создание потоков подтверждено трассировкой clone3 и снимками ps. Измерения показали замедление на малых матрицах и ускорение на больших при оговорённых условиях эксперимента. При разборе программы основное внимание потребовалось уделить различию указателя и объекта по его адресу, структурам и порядку освобождения динамической памяти.